8-5 · 최종 RAG 챗봇 개발 · 수업 전 10분

실습점검 · 환경재설정

컨테이너 열 개 · lecture_kb 11 · 가상환경

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

왜 점검하나

수업 전에 걸러 둘 것

① 이번 주에 만든 컨테이너 열 개가 모두 떠 있는지 — 하나라도 내려가면 시연이 끊긴다

② 8-3 에서 붙인 네트워크 연결 세 줄과 lecture_kb 11 조각이 그대로인지

③ 8-4 의 가상환경과 app.py 가 있는지 — 오늘 최종본의 출발점

1~3단계는 확인 · 파일은 실습에서 처음 만든다 · 재설정(5단계)은 다시 할 때만

흐름

다섯 단계 — 위에서부터 한 번씩

Airflow 는 8-4 에서 stop 해 두었다 — start 로 그대로 올라옵니다(1~4분)

1. 전부 올림	2. 네트워크·KB	3. 가상환경	4. 오늘 만들 것	5. 환경 재설정
docker start compose start	세 줄 연결 count 11	rag_env.py ls app.py	순서만 확인 아직 치지 않음	오늘 파일만 12 → 11

1단계

전부 올리세요 — 열 개

1. **docker start ollama qdrant rag-minio**
   7주차 컨테이너 셋부터

2. **cd ~/airflow-lab → docker compose start → sleep 90**
   docker compose ps 에 일곱 줄이 (healthy) 이면 된다

3. **시작이 1~4분 걸릴 수 있다**
   _PIP_ADDITIONAL_REQUIREMENTS 가 패키지를 다시 설치하느라

4. **af() { docker compose exec airflow-scheduler airflow "$@" ... }**
   < /dev/null 까지 — 이 터미널에서 쓸 줄임, 새 터미널이면 다시 만든다

확인

일곱 줄

healthy

2단계

네트워크 연결과 지식베이스를 보세요

명령	결과(실측)
docker network inspect airflow-lab_default ... | grep -E "^(ollama|qdrant|rag-minio)$"	ollama · qdrant · rag-minio 세 줄 — 8-3 에서 붙인 연결
curl -s -X POST .../collections/lecture_kb /points/count -d '{"exact": true}'	{"result":{"count":11},...} — 8-3 끝 상태
Could not resolve host 가 나면	연결이 빠진 것 — connect 다시
count 가 10 이면	8-3 의 airflow_ops.txt 가 투입구에 없다

세 줄이 다 안 나오면 8-3 수집 실습 1단계의 docker network connect 를 다시 · count 가 11 이 아니면 8-3 실습점검 6단계

3단계

챗봇 가상환경을 보세요

1. **cd ~/data-eng/week8 → source .venv/bin/activate**
   Windows(WSL2) · macOS · 리눅스 모두 같다

2. **python rag_env.py**
   PROVIDER = ollama · 모델 = llama3.2 (OpenAI 경로는 openai)

3. **ls app.py**
   8-4 의 app.py 가 오늘 최종본의 출발점 — 없으면 8-4 검색 실습 4단계

출발점

8-4 의 app.py

4단계 · 순서만

오늘 만드는 것 — 아직 치지 않는다

무엇	어디서 처음 만드나	남는가
dags/rag_pipeline_final.py	통합 실습 2단계	예 — 산출물 ①
app.py(최종본으로 덮어씀)	통합 실습 3단계	예 — 산출물 ②
dags/data/incoming/ lakehouse.txt	검증 실습 3단계	예 — lecture_kb 가 12 조각이 됨
~/data-eng/week8/submit/	검증 실습 8단계	예

산출물 둘(① 최종 DAG · ② 최종 app.py)은 검증 실습 8단계에서 submit/ 폴더로 모은다

5단계

환경 재설정 — 오늘 실습만 처음부터

1. **rm -f dags/data/incoming/lakehouse.txt**
   오늘 넣은 문서만 지우고 af dags trigger rag_pipeline_final

2. **1분 뒤 count 가 11 인지 확인**
   prune 이 lakehouse 조각을 지워 8-3 끝 모양으로 돌아온다

3. **rm -f dags/rag_pipeline_final.py · submit**
   DAG 파일은 마지막에 — 태스크가 실행할 때 다시 읽는다

4. **지식베이스 · 컨테이너는 건드리지 않는다**
   8-3 끝 모양(11)으로 되돌리는 것뿐

언제

다시 할 때만

8-5 · 수업 전 10분 마무리

다음은 최종 아키텍처 점검 (1H)

8주를 한 장으로 잇고, 배치(DAG)와 실시간(챗봇)이 lecture_kb 하나로 만나는 구조를 봅니다

- ✓ **여덟 주 — 최종 시스템에서의 자리**
- ✓ **주방(배치)과 홀(실시간)은 서로를 모른다**
- ✓ **최종본에서 더하는 것 — 적재 시각 하나**

막혔다면

그 단계 아래 「막혔어요」 · 「질문」